0. Math and Probability

Exponentials

- $a^{-1} = \frac{1}{a}$
- $(a^m)^n = a^{mn}$
- $a^m a^n = a^{m+n}$
- $e^x \geq 1 + x$

Logarithms

- **Binary log:** $\log n = \log_2 n$
- **Natural log:** $\ln n = \log_e n$
- **Exponentiation:** $\log^k n = (\log n)^k$
- **Composition:** $\log \log n = \log(\log n)$
- $a = b^{\log_b a}$
- $\log_c(ab) = \log_c a + \log_c b$
- $\log_b a^c = c \log_b a$
- **Change of Base Formula:** $\log_b a = \frac{\log_c a}{\log_c b}$
- $\log_b(1/a) = -\log_b a$
- $\log_b a = \frac{1}{\log_a b}$
- $a^{\log_b c} = c^{\log_b a}$

Stirling's approximation

- Estimate for the factorial function
- $n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n (1 + O(1/n))$
- $\log(n!) \in \Theta(n \log n)$

Summation and series

- Finite GP: $\sum_{k=0}^{n} x^k = 1 + x + x^2 + \cdots + x^n = \frac{x^{n+1}-1}{x-1}$
- Infinite GP: $\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$
- AP: $\sum_{k=0}^{n} k = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$, $\Theta(n^2)$
- Finite AP term: $T_k = a + (k-1)d$
- Finite AP sum: $S_n = \frac{n}{2}(2a + (n-1)d)$
- Finite GP term: $T_k = ar^k$
- Finite GP sum: $S_n = \frac{a(1-r^n)}{1-r}$
- Weighted Geometric Series: $\Sigma_{i=0}^{\infty} ir^i = \frac{r}{(1-r)^2}, |r| < 1$
- Harmonic series: $\sum_{i=1}^{n} \frac{1}{i} = ln(n) + \gamma + o(1) \in \theta(logn)$
- Reciprocal Log: $\sum_{k=2}^{n} \frac{1}{logk} = \theta(n/logn)$
- Harmonic over log: $\sum_{k=2}^{n} \frac{1}{klogk} = \theta(loglogn)$
- Squared log: $\sum_{k=2}^{n} \frac{1}{k(logk)^p}$
  - $p > 1$: $\theta(1)$
  - $p = 1$: $\theta(loglogn)$
  - $0 < p < 1$: $\theta((logn)^{1-p})$

Generalised Harmonic Series

$\sum_{i=1}^{n} \frac{1}{i^p}$

- $p > 1$: $\theta(1)$
- $p = 1$: $\theta(logn)$
- $0 < p < 1$: $\theta(n^{1-p})$

L'Hospital's Rule

- technique for evaluating indeterminate limits
- if $\lim_{x\to\infty} f(x) = \infty$ and $\lim_{x\to\infty} g(x) = \infty$, then:
  - $\lim_{x\to\infty} \frac{f(x)}{g(x)} = \lim_{x\to\infty} \frac{f'(x)}{g'(x)}$
- where $f'(x)$ and $g'(x)$ are the derivatives of $f(x)$ and $g(x)$.
- $n \ln n \in o(n^2)$, meaning $n \ln n$ grows strictly slower than n^2

Useful inequalities

- $1 + x \leq e^x$

Sample Space and Event

- **Sample Space (S):** The set of all possible outcomes (elementary events) of an experiment.
- **Event (A):** Any subset of the sample space.
- **Complement of an Event ($\bar{A}$):** The set of all outcomes in S that are not in A, defined as $\bar{A} = S - A$.

Probability Distribution

A probability distribution $\Pr()$ is a mapping from events to real numbers that satisfies three axioms:

1. **Non-negativity:** $\Pr(A) \geq 0$ for all events A.
2. **Normalization:** $\Pr(S) = 1$.
3. **Additivity:** $\Pr(A \cup B) = \Pr(A) + \Pr(B)$ for mutually exclusive (disjoint) events A and B.

General Probability Rules

- **Inclusion-Exclusion Principle (for two events):** $\Pr(A \cup B) = \Pr(A) + \Pr(B) - \Pr(A \cap B)$.
- **Independence:** Two events A and B are independent if and only if $\Pr(A \cap B) = \Pr(A) \cdot \Pr(B)$.

Conditional Probability

The conditional probability of event A given event B (where $\Pr(B) \neq 0$) is defined as:

- $\Pr(A|B) = \frac{\Pr(A \cap B)}{\Pr(B)}$

Bayes' Theorem
$\Pr(A|B) = \frac{\Pr(A)\Pr(B|A)}{\Pr(B)}$

The denominator $\Pr(B)$ can be expanded using the Law of Total Probability (for an event A and its complement $\bar{A}$):

$$\Pr(A|B) = \frac{\Pr(A) \cdot \Pr(B|A)}{\Pr(A) \cdot \Pr(B|A) + \Pr(\bar{A}) \cdot \Pr(B|\bar{A})}$$

Random Variable
A function that maps the outcomes in the sample space (S) to real numbers.

- The function $f(x) = \Pr(X = x)$ is the **probability density function** (or probability mass function for discrete variables) of X.

Expectation (Mean)
The expectation or mean of a random variable X is the weighted average of all possible values of X:

- $E(X) = \sum_x x \cdot \Pr(X = x)$

Linearity of Expectation
Expectation is a linear operator:

- $E(X + Y) = E(X) + E(Y)$
- $E(aX) = aE(X)$
- **For independent variables:** $E(XY) = E(X) \cdot E(Y)$

Event Operations

1. $A \cap A' = \emptyset$
2. $A \cap \emptyset = \emptyset$
3. $A \cup A' = S$
4. $(A')' = A$
5. $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$
6. $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
7. $A \cup B = A \cup (B \cap A')$
8. $A = (A \cap B) \cup (A \cap B')$
9. $(A_1 \cup A_2 \cup \cdots \cup A_n)' = A_1' \cap A_2' \cap \cdots \cap A_n'$
10. $(A \cup B)' = A' \cap B'$
11. $(A_1 \cap A_2 \cap \cdots \cap A_n)' = A_1' \cup A_2' \cup \cdots \cup A_n'$
12. $(A \cap B)' = A' \cup B'$

Permutations and Combinations
$P_r^n = \frac{n!}{(n-r)!} = n(n-1)(n-2)\ldots(n-(r-1))$
$\binom{n}{r} = \frac{n!}{r!(n-r)!}$
$\binom{n}{r} = \binom{n}{(n-r)}$

Probability

- $P(A) = P(A \cap B) + P(A \cap \overline{B})$
- $P(A_1 \cap A_2 \cap \cdots \cap A_n) = P(A_1) + P(A_2) + \cdots + P(A_n)$, if $A_i \cap A_j = \emptyset$ for any $i \neq j$
- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ ------- Principle of Inclusion and Exclusion
- If $A \subset B$, then $P(A) \leq P(B)$
- A and B are independent $\iff P(A \cap B) = P(A)P(B)$ ------- $i.e\ A \perp B$
- Suppose P(A) > 0, P(B) > 0. If A and B are independent, then A and B are not mutually exclusive.
- Suppose P(A) > 0, P(B) > 0. If A and B are mutually exclusive, then A and B are dependent.
- S and $\emptyset$ are independent of any other event.
- If A ⊥ B, then A ⊥ B′, A′ ⊥ B, and A′ ⊥ B′.
- $A \perp B \iff P(B|A) = \frac{P(A \cap B)}{P(A)} = \frac{P(A)P(B)}{P(A)} = P(B)$, if $P(A) \neq 0$
- Contained: $A \subset B$
- Equivalent: $A = B$
- Mutually Exclusive: $A \cap B = \emptyset$
- Law of Total Probability: $P(B) = \sum_{i=1}^n P(B \cap A_i) = \sum_{i=1}^n P(A_i)P(B|A_i)$
- $P(A|B) = \frac{P(A \cap B)}{P(B)}$
- $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$
- $P(A_k|B) = \frac{P(A_k)P(B|A_k)}{\sum_{i=1}^n P(A_i)P(B|A_k)}$
- $P(A|B) = 1 - P(\overline{A}|B)$
- $P(A) = P(A|B)P(B) + P(A|\overline{B})P(\overline{B})$

Bernoulli Trials
A **Bernoulli Trial** is an experiment with exactly two outcomes: success (with probability p) and failure (with probability $q = 1 - p$).

Geometric Distribution
If X is the number of independent Bernoulli trials needed to obtain the first success:

- **Probability:** $\Pr(X = k) = q^{k-1}p$
- **Expectation:** $E(X) = \frac{1}{p}$

Binomial Distribution
If X is the number of successes in n independent Bernoulli trials:

- **Probability:** $\Pr(X = k) = \binom{n}{k}p^k q^{n-k}$

---

1. Asymptotic Analysis

Big-O (g is upper bound of f): $f \in O(g)$

- Definition: $\exists c > 0 \wedge n_0 > 0 \text{ s.t } \forall n \geq n_0 : 0 \leq f(n) \leq c \cdot g(n)$
- Theorem+Proof:
  1. $lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$
  2. $lim_{n\to\infty} \frac{f(n)}{g(n)} = L, L \in \mathbb{R}$
  3. $\forall \epsilon > 0, \exists n_0 \text{ s.t } \forall n \geq n_0, \frac{f(n)}{g(n)} - L < \epsilon$

Ω (g is lower bound of f): $f \in \Omega(g(n))$

- Definition: $\exists c > 0 \wedge n_0 > 0 \text{ s.t } \forall n \geq n_0 : 0 \leq c \cdot g(n) \leq f(n)$
- Theorem+Proof:
  1. $lim_{n\to\infty} \frac{f(n)}{g(n)} > 0$
  2. $lim_{n\to\infty} \frac{f(n)}{g(n)} = L, L > 0$
  3. $lim_{n\to\infty} \frac{g(n)}{f(n)} = \frac{1}{L}$
  4. $lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$

θ (g is tight bound of f): $f \in \theta(g(n))$

- Definition: $\exists c1, c2 > 0 \wedge n_0 > 0 \text{ s.t } \forall n \geq n_0 : 0 \leq c_1 \cdot g(n) \leq f(n) \leq c2 \cdot g(n)$
- Theorem+Proof:
  1. $0 < lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$
  2. $\theta = O \cap \Omega$

Little-o (g is strict upper bound of f): $f \in o(g(n))$

- Definition: $\forall c > 0, \exists n_0 > 0 \text{ s.t } \forall n \geq n_0 : 0 \leq f(n) < c \cdot g(n)$
- Theorem+Proof:
  1. $lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$
  2. $\forall \epsilon > 0, \exists n_0 > 0 \text{ s.t } \forall n \geq n_0 : \frac{f(n)}{g(n)} < \epsilon$

ω (g is strict lower bound of f): $f \in \omega(g(n))$

- Definition: $\forall c > 0, \exists n_0 > 0 \text{ s.t } \forall n \geq n_0 : 0 \leq c \cdot g(n) < f(n)$
- Theorem+Proof:
  1. $lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$
  2. $\forall M > 0, \exists n_0 \text{ s.t } \forall n \geq n_0 : \frac{f(n)}{g(n)} > M$

Properties

1. Reflexivity

- $f(n) \in O(f(n))$
  - Choose c = 1
  - $\forall n, f(n) \leq 1 \cdot f(n)$
- $f(n) \in \Omega(f(n))$
  - Choose c = 1
  - $\forall n, f(n) \geq 1 \cdot f(n)$
- $f(n) \in \theta(f(n))$
  - Chose c1 = c2 = 1

2. Transitivity

- $f(n) \in O(g(n)) \wedge g(n) \in O(h(n)) \implies f(n) \in O(h(n))$
  - $f(n) \in O(g(n)) \Rightarrow f(n) \leq c_1 g(n)$
  - $g(n) \in O(h(n)) \Rightarrow g(n) \leq c_2 h(n)$
  - $f(n) \leq c_1 g(n) \leq c_1 c_2 h(n)$
  - Let c = c1c2
- $f(n) \in \Omega(g(n)) \wedge g(n) \in \Omega(h(n)) \implies f(n) \in \Omega(h(n))$
  - $f(n) \geq c_1 g(n) \wedge g(n) \geq c_2 h(n)$
  - $f(n) \geq c_1 c_2 h(n)$
- $f(n) \in \theta(g(n)) \wedge g(n) \in \theta(h(n)) \implies f(n) \in \theta(h(n))$
  - Prove by transitivity of O and Ω
- $f(n) \in o(g(n)) \wedge g(n) \in o(h(n)) \implies f(n) \in o(h(n))$
  - $f(n) \in o(g(n)) \implies \forall \epsilon > 0, f(n) \leq \epsilon g(n)$
  - $g(n) \in o(h(n)) \implies \forall \delta > 0, g(n) \leq \delta h(n)$
  - Choose $\epsilon = \sqrt{\eta}, \delta = \sqrt{\eta}$
  - $f(n) \leq \sqrt{\eta} g(n) \leq \eta h(n)$
- $f(n) \in \omega(g(n)) \wedge g(n) \in \omega(h(n)) \implies f(n) \in \omega(h(n))$
  - $f(n) \geq c_1 g(n), g(n) \geq c_2 h(n)$
  - $f(n) \geq c_1 c_2 h(n)$

3. Symmetry

- $f(n) \in \theta(g(n)) \iff g(n) \in \theta(f(n))$
  - From definition: $c_1 g(n) \leq f(n) \leq c_2 g(n)$
  - $\frac{1}{c_2} f(n) \leq g(n) \leq \frac{1}{c_1} f(n)$

4. Complementarity

- $f(n) \in O(g(n)) \iff g(n) \in \Omega(f(n))$
  - $f(n) \leq cg(n) \iff g(n) \geq \frac{1}{c} f(n)$
- $f(n) \in o(g(n)) \iff g(n) \in \omega(f(n))$
  - $\forall \epsilon > 0, f(n) \leq \epsilon g(n) \iff \forall c > 0, g(n) \geq cf(n)$
  - (choose c = 1/ϵ)

---

2. Recurrences and Master Theorem

Remarks:

- If $f(n) \in O(n)$, then there exists two constant $c > 0$ and $n_0 > 0$ such that $f(n) \leq cn$ if $n \geq n_0$
  - For **upper bound** calculation, we can replace $\theta(n)$ with cn
- If $f(n) \in \Omega(n)$, then there exists two constant $c > 0$ and $n_0 > 0$ such that $f(n) \geq cn$ if $n \geq n_0$
  - For **lower bound** calculation, we can replace $\theta(n)$ with cn

(1) Telescoping Series

- Find a way to cancel out each other
- e.g $T(n)/n = \frac{2}{n}T(\frac{n}{2}) + 1 = T(\frac{n}{2})/\frac{n}{2} + 1$

(2) Substitution Method

- Guess a solution, then verify by induction

(3) Recursion Tree

- Find how many levels and leaves

(4) Master Theorem

- Generic form: $T(n) = aT(n/b) + f(n)$
  - $a \geq 1$
  - $b \geq 1$
  - $f(n) \in \Omega(1)$
- Classify the work into two types and compare costs
  - Splitting/combining: $f(n)$ ==> dominant term
  - Solving base cases: Cost is linear in the **number of leaves** = n^d
- Number of leaves = n^d where $d = log_b a$ is the **critical exponent**
- Recursion Tree:
  - Tree height: $log_b n$
  - Number of children of a node: a
  - Number of leaves: $a^{log_b n} = n^{log_b a} = n^d$
- **Case 1**: $f(n) \in O(n^{d-\epsilon})$ for some constant $\epsilon > 0$
  - Work done at leaves dominates
  - Ans: $T(n) \in \theta(n^d)$
- **Case 2**: $f(n) \in \theta(n^d log^k n)$ for some constant $k \geq 0$
  - Balanced contribution at all levels
  - Ans: $T(n) \in \theta(n^d log^{k+1} n)$
- **Case 3**: $f(n) \in \Omega(n^{d+\epsilon})$ for some constant $\epsilon > 0$
  - $af(n/b) \leq cf(n)$ for some constant $c < 1$ (regularity condition)
  - Work done at root dominates
  - Ans: $T(n) \in \theta(f(n))$

---

3. Proof of correctness

Iterative algorithm

1. Loop invariant: Induction hypothesis
2. Initialisation: Base case
3. Maintenance: Inductive step
4. Termination: Proof by induction implies the correctness of the algo

Fibonacci

Proof

1. Loop invariant: At the start of iteration `i`: `prev2 = Fib(i - 2), prev1 = Fib(i - 1)`
2. Initialisation: `i = 2: prev2 = Fib(0) = 0, prev1 = Fib(1) = 1`
3. Maintenance: Suppose at iteration `i`, `prev2 = Fib(i - 2), prev1 = Fib(i - 1)`, then at end of the iteration, `prev2 = Fib(i - 1), prev1 = Fib(i)`
4. Termination: Algorithm returns `Fib(n)`

Selection sort

Proof

1. Loop invariant: $A[1..j-1]$ is sorted and ($x \leq y$ for all $x \in A[1..j-1]$ and $y \in A[j..n]$)
2. Initialisation: True because nothing in LHS
3. Maintenance: Take new element from $A[j..n]$ and swap, invariant maintained
4. Termination: New element inserted is bigger than that already inserted, and add this to the end. Thus it's sorted

Weighted Directed Graphs

Proof

Definitions

1. s: Source vertex
2. R: Set of already correct computed distances
3. $dist(u, v)$: Distance from u to v
4. $w(u, v)$: Edge cost from u to v
5. V: Set of all vertices
6. D_v: Set of all possible path distances from s to v
   1. i.e $D_v = \{\forall u \in R : d(u) + w(u, v)\}$

7. $d(v)$: $min(D_v)$

Algorithm

1. Base case: $dist(s, s) = 0$, $R = \{s\}$
2. Loop: While $R \neq V$:
    1. Select $v \in V$ such that v is not in R and v minimises D_v
    2. Update $d(v) = min(D_v)$
    3. Add v to R

Loop invariant: At the start of every iteration, $\forall u \in R, dist(s, v) = min(D_v)$

Proof of correctness of the loop invariant

1. Initialisation: $dist(s, s) = d(s) = 0$
2. Termination:
    1. At the termination point, in each iteration of the while loop, we add one extra vertex to R
    2. Every vertex $u \in R = d(u)$, and $R = V$ at the end of the computation (i.e every vertex has the shortest distance from s computed correctly)
    3. So at the end of the algorithm, all the vertices will be in R and R represents the shortest path from s to every other vertices at termination
3. Maintenance (i.e how each loop iteration maintains the invariant)
    1. At the start of the loop iteration, the algorithm chooses a new vertex v to add to R such that $min(D_v)$ is minimised (step 2.1 in the algorithm) (now to prove that this maintains the invariant)
    2. Consider the shortest path P from s to v.
        1. Visual: `s -> ... -> y -> v`
    3. Claim: Every vertex in P is in R except for v
        1. Visual: `(s -> ... -> y) -> v`
        2. Need to prove that this claim is true, because if this claim is true, then it follows that the loop invariant is maintained (i.e $dist(s, v)$ = d(v))
    4. Suppose the claim (3.3) is false:
        1. Then there exists a vertex x before v and immediately after y
            1. Visual: `(s -> ... -> y) -> x -> ... -> v`
        2. The true shortest path from s to v = $dist(s, v)$ is in D_v. So $min(D_v) \geq dist(s, v)$
        3. From the visual 3.4.1.1, we have $dist(s, v) > dist(s, y) + w(y, x)$
        4. Since y is in R, $dist(s, y) = min(D_y)$
        5. $dist(s, y) + w(y, x) = min(D_y) + w(y, x) \geq min(D_x)$
        6. Full inequality: $min(D_v) \geq dist(s, v) > dist(s, y) + w(y, x) \geq min(D_x)$
        7. Which implies $min(D_v) \geq min(D_x)$
        8. So the algorithm would've selected x instead of v. This contradicts (3.1).
    5. Thus, the claim (3.3) must be true, which implies:
        1. $min(D_v) \geq dist(s, v) = d(y) + w(y, v) \geq min(D_v)$
        2. $dist(d, v) = min(D_v)$
        3. Visual: `(s -> ... -> y) -> v`
    6. Thus, the loop invariant is satisfied ($\forall u \in R, dist(s, u) = min(D_u)$)

Efficiency of Djikstra

- The algorithm can be implemented using a priority queue

Data Structure	Insert	Extract-min	Decrease-key	Time complexity
Binary Heap	$O(logn)$	$O(logn)$	$O(logn)$	$O(\|\|E\|\| + \|\|V\|\|)log\|\|V\|\|)$
Fibonacci Heap	$O(1)$	$O(logn)$ amortized	$O(1)$ amortized	$O(\|\|E\|\| + \|\|V\|\|log\|\|V\|\|)$

Recursive Algorithms

1. Base case
2. Inductive step:
    1. Assume algo is correct for any input of size $\leq n$
    2. Show the algo is correct for any input of size n

Searching in a sorted array

1. Inductive step: Assume algo works for any input of size $< n$
2. If $x = A[mid]$, answer is YES
3. If $x > A[mid]$:
    1. $x \in A[lb..ub] \iff x \in A[mid + 1..ub]$ -- A is sorted
    2. Therefore, answer must be BinarySearch(A, mid + 1, ub, x) -- Induction Hypothesis
4. Similar for $x < A[mid]$

- Depth: $O(logn)$, $T(n) \in O(logn)$

Divide and Conquer

$$T_n = \begin{cases} \theta(1) & \text{if } n \leq 1 \\ aT(\frac{n}{b}) + f(n) & \text{otherwise.} \end{cases}$$

- a subproblems, each of size $\frac{n}{b}$
- $f(n)$ = cost of splitting and combining

Improving divide-and-conquer algorithm

1. Option 1: Reduce cost for splitting/combining
2. Option 2: Reduce number of subproblems

Exponentiation
1. $a^n = a^{n-1} \cdot a$

- $T(n) = T(n-1) + \theta(1) \in \theta(n)$

2. Fast exponentiation

- If n is even: $a^n = a^{\lfloor n/2 \rfloor} \cdot a^{\lfloor n/2 \rfloor}$
- If n is odd: $a^n = a^{\lfloor n/2 \rfloor} \cdot a^{\lfloor n/2 \rfloor} \cdot a$
- $T(n) = T(\lfloor n/2 \rfloor) + \theta(1) \in \theta(logn)$

Fibonacci with matrices

$$\begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} = \begin{pmatrix} F_n + F_{n-1} & F_n \\ F_{n-1} + F_{n-2} & F_{n-1} \end{pmatrix} = \begin{pmatrix} F_n & F_{n-1} \\ F_{n-1} & F_{n-2} \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$$

$$\begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$$

Matrix Multiplication
1. Divide and conquer
Partition matrices (A), (B), and (C) into block matrices:

$$C = \begin{pmatrix} r & s \\ t & u \end{pmatrix}, \quad A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad B = \begin{pmatrix} e & f \\ g & h \end{pmatrix}$$

Then:

$$\begin{pmatrix} r & s \\ t & u \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} e & f \\ g & h \end{pmatrix}$$

- $r = ae + bg, s = af + bh, t = ce + dg, u = cf + dh$
- All blocks $(r, s, t, u, a, b, c, d, e, f, g, h)$ are $\left(\frac{n}{2} \times \frac{n}{2}\right)$ matrices.
- 8 multiplications of $\left(\frac{n}{2} \times \frac{n}{2}\right)$ matrices
- 4 additions of $\left(\frac{n}{2} \times \frac{n}{2}\right)$ matrices
- $T(n) = 8T\left(\frac{n}{2}\right) + \Theta(n^2)$
- $T(n) \in \Theta(n^{\log_2 8}) = \theta(n^3)$

2. Strassen's Algorithm

Note

$$C = \begin{pmatrix} r & s \\ t & u \end{pmatrix}, \quad A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad B = \begin{pmatrix} e & f \\ g & h \end{pmatrix}$$

Strassen's 7 Products

- Instead of 8 multiplications, compute:

$$\begin{aligned} P_1 &= a(f-h) \\ P_2 &= (a+b)h \\ P_3 &= (c+d)e \\ P_4 &= d(g-e) \\ P_5 &= (a+d)(e+h) \\ P_6 &= (b-d)(g+h) \\ P_7 &= (a-c)(e+f) \end{aligned}$$

Expansion

$$\begin{aligned} r &= P_5 + P_4 - P_2 + P_6 \\ s &= P_1 + P_2 \\ t &= P_3 + P_4 \\ u &= P_5 + P_1 - P_3 - P_7 \end{aligned}$$

Time complexity

- 7 multiplications of $\left(\frac{n}{2} \times \frac{n}{2}\right)$ matrices
- 18 additions/subtractions of $\left(\frac{n}{2} \times \frac{n}{2}\right)$ matrices
- $T(n) = 7T\left(\frac{n}{2}\right) + \Theta(n^2)$
- $T(n) \in \Theta(n^{\log_2 7}) \in \Theta(n^{2.807\cdots})$

---

4. Sorting

Lower bound for comparison-based sorting

- **Theorem: The worst-case complexity of any comparison-based sorting algo is** $\Omega(nlogn)$
- Decision tree: binary tree with at least $n!$ leaves, each permutation a possible answer
- Height of tree at least $log(n!) \in nlogn - nloge + O(logn) \subseteq \Theta(nlogn)$ (Stirling's approx.)

Merge k sorted arrays: $\Omega(kn \log k)$

- As the k arrays are sorted, number of possible ways to combine them is $(kn)!/(n!)^k$
- $\log((kn)!/(n!)^k) \in \Omega(kn \log k)$

Non-comparison sorts
Assume: $A[i] \in 1, 2, \ldots, k$
Algorithm:

- Count occurrences
- Reconstruct sorted array
- $O(n+k)$, no comparisons required.

Quick Sort

> **Note**
>
> **Partition**
>
> 1. Choose pivot.
> 2. Partition cost: $\Theta(n)$
>
> **Worst Case**
>
> - If pivot always smallest or largest: $T(n) = T(n-1) + cn$
> - $\boxed{T(n) = \Theta(n^2)}$
>
> **Average Case**
> Assume:
>
> - All permutations equally likely
> - Elements distinct
> - Pivot uniformly random
>
> Expected recurrence: $A(n) = \frac{1}{n}\sum_{j=1}^{n}[A(j-1) + A(n-j)] + cn$
>
> - Simplifies to: $A(n) = cn + \frac{2}{n}\sum_{j=0}^{n-1} A(j)$
> - Using harmonic series: $\sum_{i=1}^{n} \frac{1}{i} \approx \ln n$
> - $\boxed{A(n) = O(n \log n)}$

Desirable Properties of Sorting

1. Stable (order preserved)
   - $A[i] = A[j], ; i < j$
   - Insertion sort: stable
   - Merge sort: stable (proper implementation)
   - Quicksort: usually not stable
2. In-place
   - Uses small extra memory.
   - Insertion sort: $O(1)$
   - Merge sort: $O(n)$
   - Quicksort: $O(\log n)$ stack

---

5. Randomized Algorithms

Verification of Matrix Multiplication

- Given $A, B, C \in \mathbb{R}^{n \times n}$.
- Check if: $AB = C$
- Naive: compute AB - $O(n^3)$ (or $O(n^{2.807})$ with Strassen)

Freivald's Algorithm

> **Note**
>
> **Idea**
>
> - Given $n \times n$ matrices A, B and C, check if $A \times B = C$
> - Matrix multiplication AxB is too expensive
> - Solution: take a vector $v = \{v_1, v_2, \ldots, v_n\}$, where each v_i is either 0 or 1 chosen uniformly at random
> - Use $ABv = Cv$ to check by first computing Bv then $A(Bv)$ to avoid expensive matrix multiplication => Reduce to $O(n^2)$ from $O(n^3)$
> - If $AB = C$, then $ABv = Cv$ is always correct
> - If $AB \neq C$, then the algorithm correctly shows that $AB! = C$ with a probability of $\geq \frac{1}{2}$
>
> **Proof of probability $\geq \frac{1}{2}$:**
>
> 1. If $AB = C$, then any vector v will result in $ABv = Cv$
> 2. If $AB \neq C$:
> - $AB \neq C$ => $AB - C = D$, where D is a non zero matrix
> - Let $Dv = u$, where u is a vector = $\{u_1, u_2, .., u_n\}$
> - Algorithm is incorrect $\iff$ $ABv = Cv$ $\iff$ $u_i = 0 \; \forall i \in [0..n]$
> - $\implies$ algorithm is correct $\iff$ $u_i = 0 \; \exists i \in [0..n]$
> - Each $u_i = d_i 1 \cdot v_1 + d_i 2 \cdot v_2 + \ldots d_i j \cdot v_j + \ldots + d_i n \cdot v_n = \ldots + d_i j \cdot v_j$
> - Fix v_j, then there is at most one choice of v_j that makes $u_i = 0$
> - If $v_j = 0$, then $u_i = 0 \implies AB \neq C \implies$ algorithm is correct
> - If $v_j = 1$, then $u_i = 0 \implies AB = C \implies$ algorithm is incorrect
> 3. So algorithm is always correct if $AB = C$, but correct with probability = $\frac{1}{2}$ if $AB \neq C$
>
> **Monte Carlo**
>
> - Since probability of getting correct is only $\frac{1}{2}$, the error probability can be reduced to at most f by repeating the algo for $t = \lceil(\log(\frac{1}{f}))\rceil$ times
> - If all t outputs $AB = C$, then $AB = C$
> - Else, $AB \neq C$

- Because if $AB \neq C$, and repeat for $t = \lceil(\log(\frac{1}{f}))\rceil$ times, then error probability $\frac{1}{2^t} \leq f$

Coupon Collector / Balls and Bins

Note

Idea

- Throw m balls into n bins independently.
- Probability a fixed bin is empty: $\left(1 - \frac{1}{n}\right)^m \leq e^{-m/n}$
- Union bound: $Pr[\exists \text{ empty bin}] \leq ne^{-m/n}$
- If $m \geq 2n \ln n$, then $Pr[\text{all bins nonempty}] \geq 1 - \frac{1}{n}$

Expected Value

- $E[X] = \sum_x x \cdot Pr(X = x)$

Markov Inequality

- If $X \geq 0, Pr(X \geq aE[X]) \leq \frac{1}{a}$
- Application:
  - $E[T] = t$
  - $Pr(T \geq 100t) \leq \frac{1}{100}$

Linearity of Expectation

- $E[X + Y] = E[X] + E[Y]$
- More generally: $E\left[\sum_i X_i\right] = \sum_i E[X_i]$
- No independence required.

Indicator Variables

- For event $\mathcal{E}$:

$$1_{\mathcal{E}} = \begin{cases} 1 & \text{if } \mathcal{E} \text{ occurs} \\ 0 & \text{otherwise} \end{cases}$$

- $E[1_{\mathcal{E}}] = Pr(\mathcal{E})$. Good for counting

Hashing

- Hash table of size n.
- Hash function: $h : U \to 1, \ldots, n$
- Chain hashing: collisions stored in linked list.
- Expected number of balls in one bin: $E[X] = \frac{m}{n}$
- Expected size of list for element's bin: $1 + \frac{m-1}{n}$
- Goal: reduce collisions

Randomized Quick Sort

Note

- Pivot chosen uniformly at random.
- Partition cost: $\Theta(n)$
- Runtime proportional to number of comparisons.

Expected Comparisons

$$X_{i,j} = \begin{cases} 1 & \text{if } a_i, a_j \text{ compared} \\ 0 & \text{otherwise} \end{cases}$$

- Then: $E[\#\text{comparisons}] = \sum_{1 \leq i < j \leq n} E[X_{i,j}]$
- Observation: $Pr[X_{i,j} = 1] = \frac{2}{j-i+1}$
- $E[\#\text{comparisons}] = \sum_{i<j} \frac{2}{j-i+1}$
- $O(n \log n)$

Theorem:

- $E[T(n)] = O(n \log n)$
- Using Markov: $T(n) = O(n \log n)$ with probability ≥ 0.99

Two Types of Randomized Algorithms

Las Vegas

- Always correct.
- Runtime bounded in expectation.
- Example: randomized quicksort.

Monte Carlo

- Runtime bounded.
- May output wrong answer with small probability.
- Example: Freivald's.

Conversion:

- Las Vegas $\to$ Monte Carlo possible.
- Reverse not generally possible.